SAPIENZA
UNIVERSITÀ DI ROMA

A Multi-Layer Abstraction Approach for RL with Non-Markovian Tasks

Ingegneria Informatica, Automatica E Gestionale "ANTONIO RUBERTI"
Engineering in Computer Science

Matteo Ventali
ID number 1985026

Advisor
Prof. Fabio Patrizi

Academic Year 2025/2026

Dedicated to my family

Contents

Chapter 1

Introduction

Chapter 2

Background

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a branch of Machine Learning (ML) concerned with learning how to make sequential decisions through interaction with an environment. Within the Machine Learning area, a general distinction is made between supervised, unsupervised, and Reinforcement Learning.
In *supervised learning*, an algorithm is trained on a labeled dataset containing examples, with the objective of learning a mapping that generalizes to samples not belonging to it. On the other hand, *unsupervised learning* works on unlabeled data and aims to identify underlying structures and patterns within the available observations [3].
Reinforcement Learning differs from both paradigms because the learner is not provided with explicit examples of the correct action to perform. Instead, an agent interacts with an (initially) unknown environment, selects actions, observes their consequences, and receives a reward signal that provides feedback about its behavior. Therefore, the goal of the agent is to learn a policy that maximizes the expected cumulative reward over time.
This type of interaction describes a *sequential decision-making problem*. Unlike prediction problems in which individual outputs can often be considered independently, in RL the consequences of a decision may extend beyond the immediate transition. An action selected at a given time may influence the state reached by the agent, the actions that will subsequently become available, and the rewards that can be obtained in the future. So, the quality of a decision cannot generally be evaluated only in terms of its immediate outcome, but must also account for its long-term consequences. In fact, in RL we often refer to trajectories (sequences of states and actions) generated by the agent's behavior during its interaction with the environment.
RL can therefore be viewed as a framework for sequential decision-making under uncertainty, where an agent must learn how its actions influence both immediate and future outcomes. To reason formally about these interactions, a mathematical model of the decision process is required. The standard framework adopted for this purpose is the Markov Decision Process (MDP), which provides a formal description of states, actions, transition dynamics, and rewards. MDPs are introduced in the following

section and will be the basis for the reinforcement learning methods discussed throughout this chapter.

2.1.1 Markov Decision Processes (MDPs)

The mathematical framework adopted for modeling sequential decision-making problems is the Markov Decision Process (MDP). An MPD is defined as a tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition probability function, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor [3].
At each discrete time step t, the environment is in a state $S_t \in \mathcal{S}$. Based on this state, the agent selects an action $A_t \in \mathcal{A}$. The execution of the action triggers the environment to transition into a new state S_{t+1} and produces a scalar reward R_{t+1}.

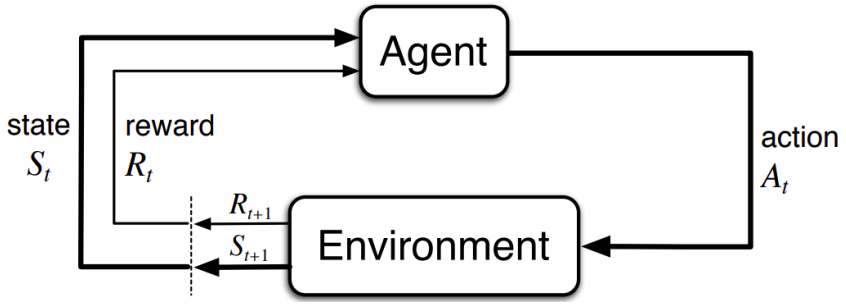

Figure 2.1. Interaction between the agent and the environment in an MDP [3].

The transition dynamics are characterized by

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{2.2}$$

which denotes the probability of reaching state s' after executing action a in state s. Looking at the P definition, it is immediate to notice that s' strictly depends only on the immediate previous state s. Therefore, with this definition, we are implicitly assuming the so-called *Markov property*: conditioned on the current state and action, the distribution of the next state is independent of the preceding interaction history. Formally, it can be expressed as follow:

$$\Pr(s' \mid s, a, S_{t-1}, A_{t-1}, \ldots, S_0) = \Pr(s' \mid s, a) \tag{2.3}$$

The current state must contain all the information relevant to predicting the future evolution of the process [3]. If additional information from the interaction history is required, the chosen state representation is *not Markovian* with respect to the problem under consideration.
The reward function assigns a numerical signal to the agent's interaction with the environment. The most general formulation corresponds to the function $R(s, a, s')$ which provides immediate feedback, whereas the agent's objective generally depends on rewards accumulated over multiple time steps. The distinction between immediate and long-term consequences is therefore central to sequential decision-making.

The discount factor γ determines the relative importance of future rewards. Values close to zero emphasize immediate outcomes, whereas values close to one assign greater importance to rewards received further in the future. The accumulation of discounted rewards will be formalized through the concept of return in the subsection on policies, returns, and value functions.
A sequence generated through interaction between the agent and the environment can be written as

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots \tag{2.4}$$

and is referred to as a *trajectory*. The probability of a trajectory is determined jointly by the transition dynamics of the MDP and the actions chosen by the agent based on its own *policy*.

2.1.2 Policies, Returns, and Value Functions

A *policy* specifies how the agent selects actions in each state. A stochastic policy π assigns a probability to every action $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s)\,. \tag{2.5}$$

For every state, these probabilities satisfy $\pi(a \mid s) \geq 0$ and $\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$. A deterministic policy is the special case in which one action is selected with probability one and can therefore be represented as a mapping $\pi : \mathcal{S} \to \mathcal{A}$.
The immediate reward obtained from a single transition is generally insufficient to evaluate an action, since its consequences may affect rewards received many steps later. The long-term outcome from time step t is represented by the *return*, defined as the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{2.6}$$

In episodic tasks, this sum terminates when a terminal state is reached. The discount factor γ controls the relative importance of immediate and delayed rewards. For continuing tasks with bounded rewards, choosing $\gamma < 1$ also ensures that the infinite sum remains finite.
Value functions measure the expected return obtained when the agent follows a given policy. The *state-value function* evaluates a state, whereas the *action-value function* evaluates the selection of a particular action in that state:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]\,, \tag{2.7}$$

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]\,. \tag{2.8}$$

The expectation accounts for both the actions selected according to π and the stochastic transitions of the environment. The state-value and action-value functions are related by

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, q_\pi(s, a). \tag{2.9}$$

An optimal policy π^* maximizes the expected return from every state. The corresponding optimal value functions are

$$v_*(s) = \max_\pi v_\pi(s), \tag{2.10}$$

$$q_*(s,a) = \max_\pi q_\pi(s,a). \tag{2.11}$$

Once q_* is known, an optimal deterministic policy can be obtained by selecting an action with maximum value:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} q_*(s,a). \tag{2.12}$$

Policies and value functions therefore provide the principal quantities used to describe and compare decision-making strategies. Their recursive structure is captured by the Bellman equations [3].

2.1.3 Bellman Equations and Value Iteration

The return in Equation (2.6) can be decomposed into the immediate reward and the discounted return from the next time step:

$$G_t = R_{t+1} + \gamma G_{t+1}. \tag{2.13}$$

This recursive relation leads to the *Bellman expectation equation*. For a policy π, the value of a state equals the expected immediate reward plus the discounted value of the successor state:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma v_\pi(s') \right]. \tag{2.14}$$

The corresponding equation for the action-value function is

$$q_\pi(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') q_\pi(s',a') \right]. \tag{2.15}$$

These equations express a consistency condition: the value assigned to the current state or action must agree with the values assigned to its possible successors. For an optimal policy, the expectation over the next action is replaced by a maximization, yielding the Bellman optimality equations

$$v_*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma v_*(s') \right], \tag{2.16}$$

$$q_*(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} q_*(s',a') \right]. \tag{2.17}$$

When the transition and reward models are known, the optimal state-value function can be computed through *value iteration*. Starting from an arbitrary estimate V_0, the Bellman optimality backup is applied repeatedly:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right]. \tag{2.18}$$

For a finite discounted MDP, the sequence of estimates converges to v_*. An optimal policy can then be extracted by acting greedily with respect to the converged values:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma v_*(s') \right]. \tag{2.19}$$

Value iteration is a model-based planning method because it requires explicit knowledge of P and R. In RL, these quantities are often unknown, motivating model-free methods such as Q-learning [3].

2.1.4 Goal MDPs and Sparse Rewards

Many RL problems are naturally formulated in terms of achieving a specific objective. In these scenarios, the agent receives a reward signal only when it reaches the final goal, and therefore it is not guided through a reward signal that continuously assesses every intermediate action. Such problems can be modeled as goal-oriented MDPs, in which the task objective is explicitly associated with a goal condition.

Let g denote a goal and let $\mathcal{G}_g \subseteq \mathcal{S}$ be the set of states that satisfy it. A simple sparse formulation assigns a positive reward only when a transition reaches a goal state:

$$R_g(s, a, s') = \begin{cases} r_g, & \text{if } s' \in \mathcal{G}_g, \\ 0, & \text{otherwise} \end{cases} \tag{2.20}$$

where $r_g > 0$ is the reward associated with successful goal achievement.
Depending on the setting, reaching a goal state may also terminate the episode.

A reward function is described as *sparse* when the agent receives informative feedback only on a small fraction of the transitions it experiences. On the other hand, when the agent receives a high frequency reward signal from the environment, we refer to *dense* reward function.

A notable sparse-reward setting is the *goal MDP*: an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a *goal MDP* if and only if there exists a set $\mathcal{G} \subseteq \mathcal{S}$ of goal states for which the reward function satisfies

$$R(s, a, s') = \begin{cases} 1, & \text{if } s \notin \mathcal{G} \text{ and } s' \in \mathcal{G}, \\ 0, & \text{otherwise} \end{cases} \tag{2.21}$$

In addition, goal states have zero optimal continuation value:

$$V^*(s) = 0, \qquad \forall s \in \mathcal{G}. \tag{2.22}$$

Under these conditions, the agent receives a unit reward only when it enters the goal set from a non-goal state. Once a goal state has been reached, no further return can be accumulated from that state.

Sparse rewards provide a direct representation of the task objective because they reward behavior that actually achieves the final goal. However, they also make learning more difficult [3, 1]. Before encountering a successful trajectory, the agent may receive the same reward for many different actions and states, with no direct indication of whether its behavior is moving toward or away from the objective. This difficulty is closely related to exploration. Without informative intermediate feedback, the agent must discover successful behavior largely through interaction with the environment. If the goal can be reached only after a long sequence of appropriate decisions, the probability of finding a successful trajectory without the reward signal's guidance may be very low.
Sparse-reward problems will be central into the framework proposed in Chapter 3.

2.1.5 Value-Based Methods

Value-based reinforcement learning methods compute an optimal policy by estimating value functions rather than representing the policy directly. In particular, action-value methods learn the expected return associated with each state-action pair and derive a policy by selecting actions adopting a greedy strategy with respect to these estimates.

Q-learning. Q-learning is a model-free, off-policy algorithm that estimates the optimal action-value function directly from observed transitions. After observing the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it applies the temporal-difference update

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right], \quad (2.23)$$

where α is the learning rate hyper-parameter. The maximization defines a greedy target policy, while experience can be collected through an exploratory behavior policy such as ϵ-greedy. Tabular Q-learning is effective for small discrete MDPs, but it becomes impractical when the state space is large because it maintains a separate estimate for every (s, a) pair [3].

Deep Q-Networks. Deep Q-Networks (DQN) address this limitation by approximating the action-value function with a neural network $Q(s, a; \theta)$. To improve training stability, DQN stores transitions in a replay memory and learns from randomly sampled mini-batches. It also maintains a target network with parameters θ^-, which are periodically synchronized with the online parameters θ. For a transition (s, a, r, s', d), the target is

$$y^{\mathrm{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \theta^-), \quad (2.24)$$

where d indicates whether the transition is terminal. The online network is trained by minimizing the squared difference between this target and $Q(s, a; \theta)$ [2].

Double Deep Q-Networks. The maximization in the DQN target can lead to overestimated action values because the same estimates are used both to select and evaluate the next action. Double Deep Q-Networks (DDQN) reduce this bias by using the online network for action selection and the target network for action evaluation:

$$y^{\mathrm{DDQN}} = r + \gamma(1-d)Q\left(s', \arg\max_{a' \in \mathcal{A}} Q(s', a'; \theta); \theta^{-}\right). \tag{2.25}$$

DDQN retains the replay memory, target network, and optimization procedure of DQN while changing only the construction of the temporal-difference target. This generally produces more reliable value estimates and is the value-based learning algorithm adopted in the proposed framework [4].

2.2 Logical Temporal Task Specification

2.2.1 Linear Temporal Logic on Finite Traces

2.2.2 Deterministic Finite Automata

2.2.3 Automata as Task-Progress Representations

2.3 Reward Shaping

2.3.1 Shaping Rewards

2.3.2 Potential-Based Reward Shaping

2.3.3 Policy Invariance

2.4 State-Space Abstraction

2.4.1 Abstract States and Transition Models

2.4.2 Abstraction Mappings

2.4.3 Multi-Level and Hierarchical Abstraction

2.5 Related Work

2.5.1 Temporal Logic and Automata-Based Reinforcement Learning

2.5.2 Reward Shaping for Non-Markovian and Sparse-Reward Tasks

2.5.3 Abstraction-Based and Hierarchical Reinforcement Learning

2.5.4 Positioning of This Thesis

Chapter 3

Proposed Framework

Chapter 4

Experimental Evaluation

Chapter 5

Conclusions and Future Work

5.1 Summary of the Approach

5.2 Main Findings

5.3 Contributions

5.4 Limitations

5.5 Future Research Directions

5.6 Concluding Remarks

Bibliography

[1] Cipollone, R., Favorito, M., Maiorana, F., De Giacomo, G., Iocchi, L., and Patrizi, F. Exploiting robot abstractions in episodic rl via reward shaping and heuristics. *Robotics and Autonomous Systems*, **193** (2025), 105116. doi:10.1016/j.robot.2025.105116.

[2] Mnih, V., et al. Human-level control through deep reinforcement learning. *Nature*, **518** (2015), 529. doi:10.1038/nature14236.

[3] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. The MIT Press, second edn. (2018).

[4] van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100 (2016).